NetraJyoti AI

Technical Product Requirements Document (Technical PRD)

A Bengali-first, deterministic safety-guidance MVP for people seeking appropriate eye-care next steps in rural West Bengal. It provides care navigation and does not diagnose eye disease.

Document intent. This technical PRD translates the capstone PRD, approved user journey, and current MVP implementation into build-ready requirements for engineering, quality assurance, operations, and review.

Document control	Value
Product	NetraJyoti AI MVP
Version	1.1 - 3 September 2026
Primary language	Bengali-first interface with smaller English support text
Client	React + TypeScript + Vite + Tailwind CSS / PWA-ready web client
Server	Java 21 + Spring Boot 3 REST API
Data	PostgreSQL clinical source, pathway, criterion, guidance, and linkage tables; JdbcTemplate retrieval for routing/RAG
Safety posture	Java fixes the route from allowlisted UI codes and matching PostgreSQL criteria; Ollama cannot select or change it
Status	Live capstone implementation baseline; pilot production controls are explicitly marked as release gates
Wireframe reference	Published live wireframe: https://rahulkarcj.github.io/netrajyoti-ai-mvp/

1. Product and technical context

NetraJyoti AI addresses delay, fear, and confusion for Bengali-speaking users who are unsure where to seek eye care. A short guided interaction produces one fixed safe route: urgent eye care, routine eye-care guidance, or human support.

The live MVP is an information and care-navigation product. It never presents a disease label, claims clinical diagnosis, or lets generative AI select or change the route.

1.1 Product objectives

- Give Bengali-speaking users a low-literacy-friendly, mobile-first way to select fixed eye-concern options.
- Use Java deterministic routing with PostgreSQL criteria to select URGENT, ROUTINE, or HUMAN_SUPPORT.
- Provide demo district/town or block selection, static demo eye-care options, family sharing, and a human-support route.
- Show selected concerns and the fixed route with bilingual, approved next-step guidance.

• Optionally use local Ollama only after the route is fixed to create a short Bengali explanation that Java validates against safe policy.

1.2 Scope and non-goals

In MVP scope	Out of scope / prohibited
Bengali-first fixed-option journey, optional browser speech/text context, deterministic route, selected concerns, static demo directions, sharing/copy, and restart.	Disease diagnosis, medication recommendation, treatment plan, triage model training, or autonomous clinical decision-making.
Demo district/town or block selection and static demo facility cards in React. PostgreSQL service_locations is reserved but not wired to the UI.	Claims that availability, hours, addresses, or facility details are live/verified unless an approved directory integration is introduced.
Optional local Ollama Bengali route explanation after PostgreSQL guidance retrieval and Java policy validation/fallback.	Sending raw free-text, recorded audio, or patient details to an external service without a separately approved privacy and consent design.
PWA-capable browser experience.	Native mobile application-store distribution in the MVP.

2. End-to-end workflow and routing

This end-to-end workflow and its screen identifiers are aligned with the published NetraJyoti wireframe for the current live capstone MVP. The wireframe is the visual representation of this flow for capstone review and demo validation. Reference: https://rahulkarcj.github.io/netrajyoti-ai-mvp/ Pilot and production readiness gaps - including clinically approved content, live facility verification, access controls, automated test coverage, and managed deployment - are documented explicitly as future release gates, not as current capabilities. The browser keeps only transient journey state, and every result offers back or restart navigation.

Stage	User/system action	Transition rule
1. Consent	User reads value and safety notice, then selects consent.	Consent enables Start safely. No patient details are requested.
2. Concerns	User selects one or more fixed concern options; voice/text is optional context.	Selections move to urgent-warning review. Free text does not select a route.
3 / 3A. Urgent and history	User answers urgent-warning questions and may select optional eye-care history.	Java evaluates only allowlisted codes. In current seed data, history is retrieved but does not change a route.

4. Safe guidance	Browser calls Java route evaluation and displays the result.	Java retrieves PostgreSQL criteria, then returns URGENT, ROUTINE, or HUMAN_SUPPORT. Urgent has precedence.
5A urgent	Immediate care advice, static demo care-direction paths, sharing, or care confirmation.	Sub-paths 5A.1-5A.6 retain access to the urgent result.
5B routine	User chooses district and town/block and sees routine visit guidance.	Step 6 shows the visit recommendation; Step 7 is the single facility-list screen.
6-7 routine	Recommendation then service direction.	No result-page audio is shown. The user can return or restart.
5C human support	User is guided to ASHA/PHC/trusted-family support.	Static area selection and nearby-support demo content may be shown; user can restart.

2.1 Deterministic routing contract

Safety invariant. The backend rules module is the sole authority for urgency. UI state, optional voice/text, AI output, patient profile fields, and service selection must not alter the urgency decision.

Route	Entry condition	Required guidance and actions
Urgent (5A)	At least one matching URGENT_IF_ANY PostgreSQL criterion, or a Java urgent fallback.	Show seek-eye-care-without-delay, selected concerns, static demo care-direction actions, family sharing, care confirmation, return, and restart.

Routine (5B-7)	No urgent criterion; one or more matching ROUTINE_IF_ANY criteria.	Offer district/town selection, visit recommendation, service direction on Screen 7, sharing/copy, return, and restart.
Human support (5C-5C.2)	Explicit needsHumanSupport request, OTHER concern, matching ESCALATE_IF_ANY criterion, or no eligible route.	Explain local health-worker, PHC, eye-clinic, or trusted-family support; offer static area-based demo options and restart.

2.2 Urgent-route state transitions

Current stage	Trigger	Next stage
5A	Find verified care	5A.1
5A.1	Use current location	5A.2
5A.1	District/block or PHC support	5A.3
5A.2	Location allowed	5A.4
5A.2	Permission declined / district option	5A.3
5A.3	District + block selected; view services	5A.4
5A	Share urgent message	5A.5
5A	Continue to care	5A.6
5A.4/5A.5/5A.6	Return urgent result	5A

3. Solution architecture

NetraJyoti is a browser-based, API-driven capstone MVP. The client sends allowlisted selection codes. Java retrieves matching PostgreSQL criteria and fixes the route before optional local Ollama creates a short Bengali explanation from PostgreSQL route guidance. Java validates the explanation or returns fixed safe content. The demo does not persist patient details.

Layer	Technology	Live responsibilities

Presentation	React, TypeScript, Vite, Tailwind CSS	Bengali-first responsive screens, transient workflow state, static demo area/facility content, Web Share/Clipboard, optional browser speech input, and PWA setup.
API	Java 21, Spring Boot 3, REST, Spring Security, Bean Validation	Validate route/explanation requests, route evaluation, PostgreSQL retrieval, Ollama invocation, safety-policy fallback, CORS, and health endpoint.
Safety domain	SafetyRoutingService plus deterministic Java fallback	Uses retrieved clinical criteria to fix URGENT, ROUTINE, or HUMAN_SUPPORT. UI detail, RAG content, and Ollama output cannot change it.
Persistence	PostgreSQL with JdbcTemplate; JPA ServiceLocation repository reserved	clinical_source, clinical_pathway, clinical_criterion, clinical_guidance, and clinical_pathway_source support demo routing and RAG. service_locations is not integrated into the live UI.
AI integration	Local Ollama REST /api/chat, called only from Java	After route lock, retrieves route guidance from PostgreSQL, requests short Bengali explanation, validates it, or returns fixed safe fallback.
Delivery	Docker Compose for PostgreSQL 16 and Ollama; local Spring Boot and Vite dev servers	Current capstone run mode. Managed TLS hosting, backups, deployment controls, and production observability are future release gates.

3.1 Context data flow

- The browser sends only fixed concern and optional history codes plus an explicit human-support request. The route API returns a deterministic route and rule version.
- District and town/block selections are currently used in the browser to display static demo service-direction content; the live UI does not call a location or service API.
- The route-explanation request is optional, server-side only, and eligible after any fixed route. AI_SUMMARY_ENABLED and DEMO_RAG_ENABLED are evaluated by Java; Ollama is never called from the browser.
- If Ollama is disabled, unavailable, times out, fails validation, or produces unsafe text, Java returns deterministic fallback guidance. The route and user journey remain complete.

4. Frontend requirements

Area	Technical requirement
State model	Use a typed workflow state machine or reducer. Persist only low-risk, session-scoped UI state if needed; reset on “Start again”.
Internationalisation	Bengali is the primary label and English is secondary, visually smaller text. Avoid untranslated utility labels on user-facing screens.
Accessibility	Large touch targets (minimum 44px), high contrast, clearly visible selection states, keyboard focus states, semantic buttons/labels, status announcements, and readable bilingual hierarchy.
Progression	Every stage must expose an understandable primary action and a safe back/return/restart route where appropriate. The wireframe “Entered from / Trigger” traceability is documentation, not an end-user control.
Voice	Use browser speech recognition only as optional input convenience. Display captured text for confirmation. Failure/unavailability must leave typed/fixed-option flow fully usable.
Sharing	Use Web Share API where supported; otherwise offer Copy. Provide clear success/error status. Never infer that sharing completed successfully.
PWA	Installable manifest, responsive layout, HTTPS-only deployment, cache only static application assets; do not cache API responses containing patient data.

5. Backend services and API contract

The live REST API uses JSON and UTF-8 under /api/v1. It accepts allowlisted codes, performs Bean Validation, and returns standard Spring error responses. The table below documents the implemented contract; service-directory and client-feature APIs remain future work.

Method and path	Purpose	Implemented request / response
POST /api/v1/routing/evaluate	Evaluate the deterministic safety route.	Request: concerns[], history[], needsHumanSupport. Response: outcome (URGENT|ROUTINE|HUMAN_SUPPORT), ruleVersion. Unknown enum values are rejected by validation.

POST /api/v1/ai/route-explanation	Return an optional safe Bengali explanation after route lock.	Request: outcome, concerns[], history[]. Response: summaryBn, source (OLLAMA_RAG_VALIDATED|SAFE_FIXED_FALLBACK), sources[].
GET /actuator/health	Local health check.	Actuator health response. The endpoint is public in the capstone configuration.
Not implemented in live API	Locations, service directory, client feature flags, caregiver-summary, administrative, and audit APIs.	The current frontend uses static demo data for locations and facilities. These APIs remain pilot/production work.

5.1 Routing request and response rules

API validation. Reject missing or unknown fixed identifiers; never route based on arbitrary free text. Return HTTP 400 with a stable error code for invalid input. Return HTTP 429 for rate limits and a safe generic fallback message for unexpected failure.

Field	Live rules
concerns	Array of approved Concern enums sent from fixed UI options. May be empty only for explicit human support / fallback journeys.
history	Optional HistoryCode enums. Java includes them in PostgreSQL criterion lookup, but current demo seed data has no history criterion that changes the route.
needsHumanSupport	Explicit boolean request. A true value returns HUMAN_SUPPORT unless an urgent condition has precedence.
outcome	One of URGENT, ROUTINE, HUMAN_SUPPORT. It is fixed by Java before the route-explanation request.
route-explanation input	Route plus fixed concern/history arrays only. Browser speech/transcript, patient details, location, and free text are not sent to Ollama.

6. Data model

Live structure	Key fields / purpose	Implementation note
clinical_source	id, organization, title, url, accessed_on	Provenance records for demo clinical knowledge.
clinical_pathway	id, route, status, title	Defines the demo route pathway.
clinical_criterion	pathway_id, criterion_type, input_code	Routing criteria: URGENT_IF_ANY, ROUTINE_IF_ANY, and ESCALATE_IF_ANY.

clinical_guidance	pathway_id, language, content	Bengali/English guidance retrieved after route lock for RAG context.
clinical_pathway_source	pathway_id, source_id	Links pathway to the displayed reference source.
service_locations	JPA entity/repository placeholder	Present in the schema/codebase but not queried by the current frontend workflow.
No current tables	Feature flags, patient records, audit events, AI request audit, or analytics events	Flags are environment/configuration properties. Do not represent these future controls as implemented.

7. Controlled AI / GenAI design

Controlled AI is an optional explanation layer, not a routing mechanism. In the live MVP, local Ollama can produce a short Bengali explanation after Java has determined any route. Java safety policy accepts safe content or returns a fixed fallback.

7.1 Feature flag and decision flow

Step	Implemented behavior
1. Route first	Java runs SafetyRoutingService and fixes the route using UI codes plus matching PostgreSQL criteria. Ollama is called only afterwards.
2. Check configuration	AI_SUMMARY_ENABLED and DEMO_RAG_ENABLED are read server-side. If disabled, Java returns safe fixed fallback content.
3. Retrieve RAG context	ApprovedClinicalKnowledgeRepository retrieves route-specific guidance and source records from PostgreSQL. Retrieval cannot alter the route.
4. Generate	Java calls local Ollama /api/chat using OLLAMA_BASE_URL and OLLAMA_MODEL (default llama3.2:3b), with short Bengali output instructions.
5. Validate	CaregiverSummaryPolicy validates the output for safe, route-consistent Bengali wording and rejects unsafe/unusable text.
6. Fallback	Connection/read timeout, API error, empty/malformed response, disabled feature, or safety rejection returns SAFE_FIXED_FALLBACK.
7. Present	The result UI shows guidance based on approved information, source attribution, the selected route/concerns, and route actions. Result-page audio is intentionally absent.

7.2 AI guardrails

- Prompt and policy: do not diagnose, name a disease, prescribe medicine, change urgency, invent facilities, or claim a clinician reviewed the user.

- The live call requests short Bengali text rather than requiring structured JSON. Defensive parsing accepts text or a summary field, then Java policy validation determines acceptance.
- Post-generation policy validation enforces Bengali text, a short length, prohibited diagnosis/treatment terms, and route-consistent safe language.
- Implemented resilience is an explicit connection/read timeout and deterministic Java fallback. Retry, circuit breaker, and rate limiting are pilot release gates, not current behavior.
- The demo logs technical routing data, including selected codes and matched criteria. Before any pilot, logs must be reduced to non-content metadata and reviewed under a privacy policy.

Feature flag. ENABLE_AI_SUMMARY is configured through environment/application configuration and exposed to the client only as a boolean capability. Turning it off must need no client redeploy and must force the approved fallback.

8. Security, privacy, and safety controls

Control area	Current implementation and release gap
Secrets	Database credentials, CORS origin, Ollama configuration, and AI/RAG flags are environment variables. No OpenAI key exists in the live implementation.
Transport	Local development uses HTTP. HTTPS, secure headers, and hardened deployment configuration are production release gates.
Authentication	Routing, AI explanation, and health endpoints are public in the demo. Admin roles and protected content/service updates are not implemented.
Patient details	The current workflow does not persist patient records. Keep name, age, address, phone, audio, and raw free text out of backend requests and logs.
Location	The UI uses selected district/town/block and static demo cards. Browser geolocation and live directory integration are not live capabilities.
Content governance	Database records carry pathway/source relationships, but current content is demo-simulated and not clinically approved. Formal approval/version workflow is a pilot gate.
Auditability	Current logs include routing codes and matched criteria. Before pilot, reduce to non-content metadata, define retention, and add governed audit records.

9. Non-functional requirements

Category	Current state / release gate
Availability	Core routing and fixed fallback do not require Ollama. PostgreSQL is required for the current criteria/guidance implementation.
Performance	Routing is local database lookup; Ollama connection timeout is 10 seconds and read timeout 75 seconds. A <=10-second explanation-or-fallback target is a pilot gate.
Reliability	Browser workflow retains journey state during navigation and exposes back/return/restart controls. Refresh/close clears state.

Mobile usability	Responsive, Bengali-first UI with consistent controls. Formal device/accessibility acceptance testing remains a release gate.
Accessibility	Large selectable controls and bilingual hierarchy are implemented. WCAG 2.1 AA audit and screen-reader test evidence are pending.
Observability	Spring logs and Actuator health are available. Correlation IDs, dashboards, AI latency metrics, and privacy-safe audit telemetry are pending.
Maintainability	Typed frontend models and Java enums/services exist. No DB migration tool, linting evidence, integration suite, or e2e suite is currently established.
Data retention	No persistent patient data is designed for the demo. A formal retention/deletion policy is required before any future data collection.

10. Test strategy and acceptance criteria

Test level	Current coverage / gap
Java unit tests	SafetyRoutingServiceTest covers deterministic precedence/routing and CaregiverSummaryPolicyTest covers acceptance/fallback policy.
Spring integration tests	Not currently present. Add PostgreSQL repository, endpoint validation, CORS, health, and fallback integration coverage before pilot.
Frontend tests	No React/Vitest test suite is currently present. Dependencies do not constitute executed coverage.
End-to-end journeys	No Playwright/Cypress suite is currently present. Manually verify urgent, routine, human support, back/restart, and fallback journeys for demo.
Manual content review	Required before pilot because displayed guidance and sources are currently demo-simulated, not clinically approved.
Security checks	Secret scan, dependency remediation, CORS validation, rate limiting, log review, and OWASP-relevant API testing are release gates.

10.1 Release acceptance criteria

- Every fixed symptom/warning-sign test maps to a reviewed expected route, with no AI dependency.
- An urgent result can reach service discovery, location permission, district/block selection, facilities, sharing, care confirmation, and restart without a dead end.
- A routine result can reach district/town selection, visit guidance, service direction, caregiver summary preview, share/copy, and restart.
- A human-support result can reach ASHA/PHC/family support and area-based nearby-support options.
- With ENABLE_AI_SUMMARY=false, the Step 8 approved Bengali fallback is shown. With it true, unsafe/unavailable AI output still returns the same safe fallback.
- No view claims diagnosis, treatment recommendation, or verified real-time service availability unless supported by approved data.
- The deployed frontend and backend run over HTTPS, no secret is included in the client build, and the health endpoint is monitored.

11. Deployment and operating model

Environment	Live deployment / release requirement
Local development	Docker Compose starts PostgreSQL 16 and Ollama. Spring Boot runs locally with environment variables; Vite runs the frontend. No .env.example is committed.
Test / staging	Not currently configured. Before pilot, use separate database/secrets, controlled content, and end-to-end smoke tests.
Production	Not currently deployed as a managed production stack. Add TLS, backups, restricted admin access, monitoring, and rollback before production.
Configuration	DATABASE_URL, DATABASE_USERNAME, DATABASE_PASSWORD, CORS_ORIGIN, AI_SUMMARY_ENABLED, DEMO_RAG_ENABLED, OLLAMA_BASE_URL, and OLLAMA_MODEL are environment-specific.
Database migration	schema.sql and data.sql initialize the demo. Add Flyway or Liquibase and governed migration/version practices before production.

12. Delivery plan and ownership

Workstream	Live deliverable / next action	Primary owner
Safety and content	Demo pathways, criteria, guidance, and source records. Approve content and governance before pilot.	Clinical/content reviewer
Frontend	Responsive bilingual journey, static service cards, share/copy, optional speech input, and back/restart. Add regression tests and accessibility audit.	Frontend engineer
Backend	Routing and explanation endpoints, PostgreSQL retrieval, Ollama policy/fallback, and health. Add integration tests and production hardening.	Java backend engineer
Data and operations	Demo PostgreSQL schema/seeds and Compose runbook. Add migrations, backups, access controls, and live service-directory integration.	Platform owner
Quality assurance	Existing Java unit tests and manual demo checks. Establish route regression matrix and mobile/e2e evidence before pilot.	QA engineer
Outreach operations	No live facility verification process is implemented. Establish verification, update cadence, and escalation contacts before pilot.	Operations owner

Appendix A. Technical decision summary

The MVP uses Java deterministic routing because urgency is the most consequential decision. React/Vite provides Android-friendly browser access. PostgreSQL stores demo clinical pathways, criteria, sources, and route guidance. Local Ollama is isolated behind Java, PostgreSQL RAG retrieval, safety policy validation, feature flags, and fixed fallback content. The core route remains available when Ollama is unavailable. All current database content is demo-simulated and not clinically approved.